

ML Pricing Model

Technical Overview

Prediction Markets — Independent Probability Estimation

1. Purpose

Prediction markets like Polymarket and Gemini aggregate public information into a price. That price is a reasonable estimate of probability — but it is not always correct. Markets can be thin, manipulated, slow to update, or driven by uninformed flow.

The goal of this system is to build an independent probability model that forms its own view on a market's outcome — using Polymarket's pricing data, order book microstructure, and external news and social sentiment as inputs. This model's estimate is then compared against both Polymarket and Gemini prices to identify mispricings worth betting on.

→ This is a pricing model, not a signal validator. The previous approach used Polymarket's price as ground truth and asked whether to trust it. This approach asks: what do we think the true probability is — independent of what any market currently says?

2. High-Level Workflow

1. Identify a market on Polymarket
2. Collect all available signal for that market:
 - Polymarket price history + live order book
 - Market microstructure (spread, OBI, Kyle's lambda, VPIN)
 - News sentiment (GDELT)
 - Social sentiment (Reddit)
 - Polymarket comment discussion
3. Feed all signal into the ML model
4. Model outputs: $P(\text{YES}) = \text{e.g. } 62\%$ + Confidence score = e.g. 0.81
5. Compare model's 62% against:
 - Polymarket current price (e.g. 55%) → 7-point gap
 - Gemini current price (e.g. 51%) → 11-point gap

```
6. If confidence > threshold AND gap > minimum edge:  
    → Bet on whichever platform has the largest mispricing
```

✓ Unlike the previous approach, you are no longer limited to betting only on Gemini. Because the model forms its own independent view, any platform where the price deviates from your estimate is a potential opportunity — including Polymarket itself.

3. Prediction vs Confidence Score

This is the most important conceptual distinction in the system. They are two separate outputs that answer two different questions.

	Prediction	Confidence Score
Question it answers	What will happen?	How much should I trust my own prediction?
What it represents	P(YES) — e.g. 62%	Reliability of that 62% — e.g. 0.81
What drives it	Signal content (news, price, sentiment)	Signal quality, consistency, and model stability
Range	0% to 100%	0 to 1
When it's high	Evidence points strongly to YES	Lots of consistent, high-quality data backed the prediction
When it's low	Evidence points strongly to NO	Sparse data, conflicting signals, or unstable model output

Why the prediction alone is not enough

Consider two markets where the model outputs 62%:

	Market A — Trump impeachment	Market B — Obscure local referendum
News articles today	50+ from major outlets	0
Reddit activity	High — active discussion	None
Polymarket volume	\$2M, deep order book	\$500, 2 traders
Signal available	Rich, consistent	Almost none
Model prediction	62%	62%
Confidence score	0.88 — bet	0.21 — skip

Both markets produce the same 62% prediction. But Market A's estimate is built on rich, consistent data across multiple independent sources. Market B's 62% is essentially a guess — the model had almost no signal and is extrapolating from nothing. The confidence score captures this difference; the prediction alone cannot.

The three things confidence measures that prediction doesn't

1. Input quality

Did the model have enough data to form a reliable view? Low volume, no news, and no social discussion means the model is working with almost no information — even if it produces a specific number, that number has very low reliability.

2. Internal consistency

Do all the signals agree with each other? The model might average multiple conflicting signals into a single 62% estimate — but if news implies 70%, microstructure implies 65%, and social sentiment implies 40%, the signals are pulling in opposite directions. That disagreement is a red flag the prediction alone hides.

3. Model stability (uncertainty quantification)

Using MC Dropout — run the model 50 times with slight random variations. If it consistently outputs 61–63%, the model is stable and confident in its own answer. If it outputs anywhere from 45% to 78%, the model itself is uncertain. The spread of those 50 outputs becomes a direct measure of confidence.

```
# MC Dropout — run 50 forward passes
predictions = [model(features, training=True) for _ in range(50)]

mean = average(predictions)    # → your prediction e.g. 62%
std  = stdev(predictions)      # → model uncertainty

# Tight cluster (std=0.02) → high confidence
# Wide spread (std=0.14) → low confidence, consider skipping
```

! Intuition: the prediction is what you think will happen. The confidence score is how sure you are that you're right. You can be highly confident in a 30% prediction (clear evidence pointing to NO) or have low confidence in a 70% prediction (lots of noise, conflicting signals). They are independent.

4. Model Inputs

All inputs are collected per market. The model learns which inputs are most predictive from historical resolved markets during training.

4.1 Polymarket Pricing & Microstructure

Input	Source	What it captures
Current mid-price	CLOB WebSocket (PriceChanges)	Market's current probability estimate
Price history (OHLC)	Data API — prices-history	How the market has moved over time
Bid-ask spread	CLOB WebSocket (AggOrderbook)	Market depth and competitive health
Order Book Imbalance (OBI)	CLOB WebSocket (AggOrderbook)	Current buying vs selling pressure balance
Kyle's lambda	Data API — trades	Price impact per unit of order flow
VPIN	Data API — trades	Whether recent moves are from informed traders
HHI concentration	Data API — positions	Whale dominance / manipulation risk
Hurst exponent	Data API — prices-history	Whether price trend is persistent or noisy
GARCH volatility	Data API — prices-history	Current volatility regime vs historical baseline

4.2 News Sentiment

Input	Source	What it captures
Headline sentiment score	GDELT → FinBERT	Positive/negative/neutral tone of recent news
News volume spike	GDELT	Sudden increase in coverage signals a real event
Source credibility weighting	GDELT tone + outlet tier	Reuters/Bloomberg articles weighted more heavily than blogs

4.3 Social Sentiment

Input	Source	What it captures
Reddit post/comment sentiment	Reddit API → FinBERT	Crowd opinion on outcome

Input	Source	What it captures
Reddit volume spike	Reddit API	Sudden increase in discussion signals attention shift
Polymarket comment sentiment	Gamma API — comments	Sentiment from traders directly involved in this market

5. Model Architecture

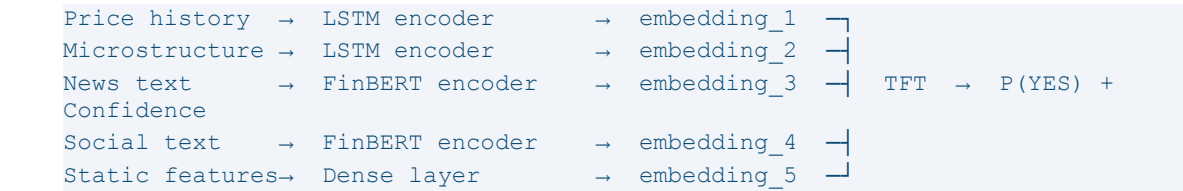
The model is a multimodal fusion architecture. Different input types require different encoders before being combined into a single prediction.

5.1 Encoders per modality

Modality	Encoder	Why
Price history + microstructure metrics	LSTM (Long Short-Term Memory)	Captures temporal patterns — how metrics evolve over time matters, not just current values
News headlines (text)	FinBERT	Pre-trained financial NLP model — converts text to sentiment scores
Reddit / comment text	FinBERT	Same text-to-sentiment pipeline as news
Static features (HHI, current spread, volume)	Dense layer	No temporal component needed — just feed directly

5.2 Fusion layer

Each encoder produces a latent embedding. A Temporal Fusion Transformer (TFT) combines them — it learns to weight which modality is most informative for each market at each moment. For example, microstructure is weighted heavily in high-volume markets; sentiment is weighted heavily when a news event has just occurred.



5.3 Outputs

Output	Type	Meaning
P(YES)	Float 0–1	Model's independent probability estimate for the market resolving YES
Confidence score	Float 0–1	Reliability of P(YES) based on input quality, signal consistency, and model stability

6. Training

The model is trained on historical resolved Polymarket markets where the true outcome is known.

6.1 Label

Binary: 1 if the market resolved YES, 0 if NO. The model learns to predict this from features collected at various points before resolution.

6.2 Features

All inputs described in Section 4, collected at multiple time snapshots before resolution (e.g. 7 days out, 3 days out, 1 day out, 1 hour out). This lets the model learn how signals evolve as resolution approaches.

6.3 Loss function

Binary cross-entropy — standard for probability estimation tasks. Calibration is applied post-training using Isotonic Regression to ensure the model's 70% prediction is correct approximately 70% of the time historically.

6.4 Evaluation metrics

Metric	What it measures
Brier Score	Mean squared error of probability estimates vs actual outcomes. Lower is better.
Calibration (Reliability diagram)	Does 70% confidence mean correct 70% of the time?
Resolution (Sharpness)	Does the model actually differentiate — or does it always predict 50%?
ROI on historical bets	At threshold X, what would the historical return have been?

7. Betting Decision Logic

Once the model outputs a prediction and confidence score, the betting decision is a simple filter:

```
model_prediction = 0.62 # our P(YES)
confidence       = 0.81 # how much we trust our 0.62

polymarket_price = 0.55 # Polymarket's current price
gemini_price     = 0.51 # Gemini's current price

poly_edge = model_prediction - polymarket_price # = 0.07
gem_edge  = model_prediction - gemini_price    # = 0.11

if confidence > THRESHOLD:                    # e.g. 0.75
    best_platform = argmax(poly_edge, gem_edge) # → Gemini
    if best_edge > MIN_EDGE:                    # e.g. 0.05
        place_bet(platform=best_platform, side=YES)
```

Parameter	What it controls	Typical starting value
THRESHOLD	Minimum confidence required to bet at all	0.75–0.85
MIN_EDGE	Minimum gap between model and market price	0.05 (5 points)
Bet sizing	How much to stake — scales with edge size and confidence	Kelly criterion or fixed fraction

! Both parameters should be calibrated by backtesting on historical resolved markets. Start conservative (high threshold, large min edge) and loosen only after validating on out-of-sample data.

8. Comparison to Previous Approach

	Previous approach	Current approach
Core idea	Trust Polymarket, verify it's reliable	Form independent probability estimate
Prediction source	Polymarket's price IS the prediction	ML model generates its own P(YES)

	Previous approach	Current approach
Confidence score purpose	Is Polymarket trustworthy?	Is our model's estimate reliable?
Where you can bet	Gemini only (when it lags Poly)	Polymarket or Gemini — wherever mispricing exists
Can you bet against Polymarket?	No	Yes — if model disagrees with Poly and confidence is high
Complexity	Lower — model only needs to score signal quality	Higher — model must learn to predict outcomes
Data required	Current market state only	Historical resolved markets for training
Upside	Simpler, faster to build	Larger opportunity set, more alpha potential

9. Build Order

1. **Data pipeline first** — get all APIs and WebSockets live and storing data. You need historical data before you can train anything.
2. **Feature engineering** — compute microstructure metrics (spread, OBI, lambda, VPIN, Hurst, GARCH) from raw data. Verify they look sensible.
3. **Sentiment pipeline** — GDELT + Reddit + Polymarket comments → FinBERT scores. Verify sentiment scores correlate with obvious events.
4. **Baseline model** — start with logistic regression or XGBoost on tabular features only (no text). Establish a performance baseline with Brier Score.
5. **Add text modality** — incorporate FinBERT embeddings. Measure whether Brier Score improves.
6. **Full architecture** — LSTM encoders + TFT fusion. Train on full historical dataset.
7. **Calibration** — apply Isotonic Regression. Verify reliability diagram. Ensure 70% prediction = ~70% historical accuracy.
8. **Backtesting** — simulate historical bets at various threshold and min-edge settings. Find parameters that produce positive ROI on out-of-sample data.
9. **Live deployment** — run model in paper-trading mode first. Compare live predictions to actual resolutions before committing real capital.

10. Summary

What we're building: An independent ML pricing model that estimates the true probability of a prediction market resolving YES, using Polymarket data, market microstructure, and external news and social sentiment as inputs.

What it outputs: Two numbers — a probability estimate $P(\text{YES})$ and a confidence score. The prediction says what we think will happen. The confidence score says how much to trust our own prediction.

How we make money: When our model's estimate diverges from the market price on Polymarket or Gemini, and confidence is above threshold, we bet on the platform with the largest mispricing.

Why this is better than trusting Polymarket's price: It opens up a larger opportunity set. We can bet on Polymarket itself when it's wrong, not just Gemini. And we can disagree with the market — which is where the real alpha lives.